Project : Analyzing the trends of COVID-19 with Python

Step 1:- Import Libraries

```
!pip install prophet
```

```
Requirement already satisfied: prophet in /usr/local/lib/python3.12/dist-packages (1.3.0)
Requirement already satisfied: cmdstanpy>=1.0.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (1.3.0)
Requirement already satisfied: numpy>=1.15.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (2.0.2)
Requirement already satisfied: matplotlib>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from prophet) (3.10.0)
Requirement already satisfied: pandas>=1.0.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (2.2.2)
Requirement already satisfied: holidays<1,>=0.25 in /usr/local/lib/python3.12/dist-packages (from prophet) (0.97)
Requirement already satisfied: tqdm>=4.36.1 in /usr/local/lib/python3.12/dist-packages (from prophet) (4.67.3)
Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages (from prophet) (7.1.0)
Requirement already satisfied: stanio<2.0.0,>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from cmdstanpy>=1.0.4->proph
Requirement already satisfied: python-dateutil<3,>=2.9.0.post0 in /usr/local/lib/python3.12/dist-packages (from holidays<1,>
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (0.
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet) (11.3.
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.4->prophet) (2025.2
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.4->prophet) (2026
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<3,>=2.9.0.post0->hc
```

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from prophet import Prophet
```

Step 2 :- Load Dataset

```
df = pd.read_csv("/content/sample_data/covid_19_clean_complete.csv")
df
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed	Deaths	Recovered	Active	WHO Region
0	NaN	Afghanistan	33.939110	67.709953	2020-01-22	0	0	0	0	Eastern Mediterranean
1	NaN	Albania	41.153300	20.168300	2020-01-22	0	0	0	0	Europe
2	NaN	Algeria	28.033900	1.659600	2020-01-22	0	0	0	0	Africa
3	NaN	Andorra	42.506300	1.521800	2020-01-22	0	0	0	0	Europe
4	NaN	Angola	-11.202700	17.873900	2020-01-22	0	0	0	0	Africa
...	...	...	...	...	...	...	...	...	...	...
49063	NaN	Sao Tome and Principe	0.186400	6.613100	2020-07-27	865	14	734	117	Africa
49064	NaN	Yemen	15.552727	48.516388	2020-07-27	1691	483	833	375	Eastern Mediterranean

Next steps: Generate code with df New interactive sheet

```
df.head()
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed	Deaths	Recovered	Active	WHO Region
0	NaN	Afghanistan	33.93911	67.709953	2020-01-22	0	0	0	0	Eastern Mediterranean
1	NaN	Albania	41.15330	20.168300	2020-01-22	0	0	0	0	Europe
2	NaN	Algeria	28.03390	1.659600	2020-01-22	0	0	0	0	Africa

Next steps: Generate code with df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 49068 entries, 0 to 49067
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   Province/State  14664 non-null  object 
 1   Country/Region  49068 non-null  object 
 2   Lat             49068 non-null  float64
 3   Long            49068 non-null  float64
 4   Date            49068 non-null  object 
 5   Confirmed       49068 non-null  int64  
 6   Deaths          49068 non-null  int64  
 7   Recovered       49068 non-null  int64  
 8   Active          49068 non-null  int64  
 9   WHO Region      49068 non-null  object 
dtypes: float64(2), int64(4), object(4)
memory usage: 3.7+ MB
```

```
df.shape
```

```
(49068, 10)
```

Step 3 :- Data Cleaning

```
df.rename(columns={"Province/State":"State","Country/Region":"Country"},inplace=True)
df
```

	State	Country	Lat	Long	Date	Confirmed	Deaths	Recovered	Active	WHO Region
0	NaN	Afghanistan	33.939110	67.709953	2020-01-22	0	0	0	0	Eastern Mediterranean
1	NaN	Albania	41.153300	20.168300	2020-01-22	0	0	0	0	Europe
2	NaN	Algeria	28.033900	1.659600	2020-01-22	0	0	0	0	Africa
3	NaN	Andorra	42.506300	1.521800	2020-01-22	0	0	0	0	Europe
4	NaN	Angola	-11.202700	17.873900	2020-01-22	0	0	0	0	Africa
...	...	...	...	...	...	...	...	...	...	...
49063	NaN	Sao Tome and Principe	0.186400	6.613100	2020-07-27	865	14	734	117	Africa
49064	NaN	Yemen	15.552727	48.516388	2020-07-27	1691	483	833	375	Eastern Mediterranean

Next steps: Generate code with df New interactive sheet

```
# Convert date
df['Date'] = pd.to_datetime(df['Date'])
```

```
df_global = df.groupby('Date')[['Confirmed','Recovered','Deaths']].sum().reset_index()
df_global
```

	Date	Confirmed	Recovered	Deaths
0	2020-01-22	555	28	17
1	2020-01-23	654	30	18
2	2020-01-24	941	36	26
3	2020-01-25	1434	39	42
4	2020-01-26	2118	52	56
...	...	...	...	...
183	2020-07-23	15510481	8710969	633506
184	2020-07-24	15791645	8939705	639650
185	2020-07-25	16047190	9158743	644517
186	2020-07-26	16251796	9293464	648621
187	2020-07-27	16480485	9468087	654036

188 rows × 4 columns

Next steps: Generate code with df_global New interactive sheet

```
df_country = df.groupby(['Country'])[['Confirmed','Recovered','Deaths']].sum().reset_index()
df_country
```

	Country	Confirmed	Recovered	Deaths
0	Afghanistan	1936390	798240	49098
1	Albania	196702	118877	5708
2	Algeria	1179755	755897	77972
3	Andorra	94404	69074	5423
4	Angola	22662	6573	1078
...	...	...	...	...
182	West Bank and Gaza	233461	61124	1370
183	Western Sahara	901	648	63
184	Yemen	67180	23779	17707
185	Zambia	129421	83611	2643
186	Zimbabwe	50794	12207	881

187 rows × 4 columns

Next steps: Generate code with df_country New interactive sheet

```
# Sort Values by Date
df_global.sort_values('Date')
```

	Date	Confirmed	Recovered	Deaths
0	2020-01-22	555	28	17
1	2020-01-23	654	30	18
2	2020-01-24	941	36	26
3	2020-01-25	1434	39	42
4	2020-01-26	2118	52	56
...	...	...	...	...
183	2020-07-23	15510481	8710969	633506
184	2020-07-24	15791645	8939705	639650
185	2020-07-25	16047190	9158743	644517
186	2020-07-26	16251796	9293464	648621
187	2020-07-27	16480485	9468087	654036

188 rows × 4 columns

Step 4:- Basic Data Understanding (EDA)

```
# Total Global Impact
df_global[['Confirmed','Deaths','Recovered']].sum()
```

	0
Confirmed	828508482
Deaths	43384903
Recovered	388408229

dtype: int64

```
# Latest Data
latest = df_global[df_global['Date'] == df_global['Date'].max()]
latest.head(10)
```

	Date	Confirmed	Recovered	Deaths
187	2020-07-27	16480485	9468087	654036

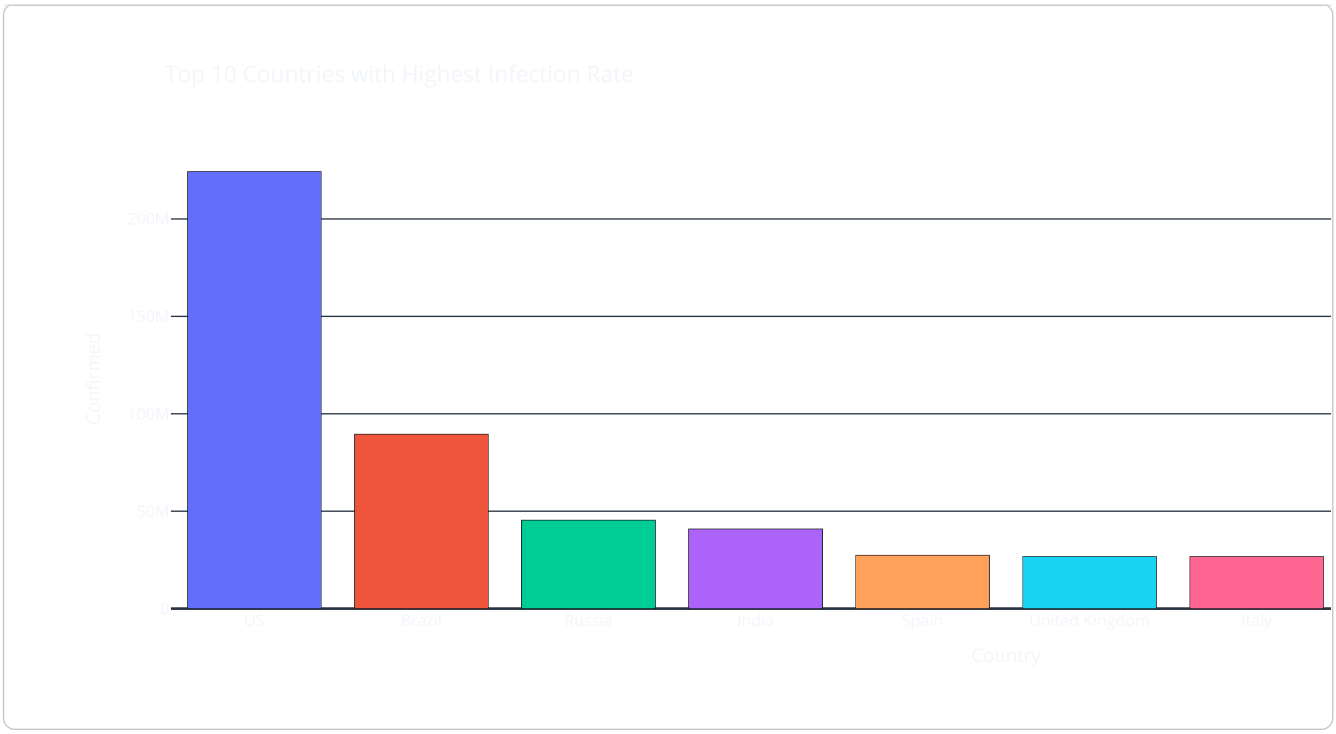
Step 5:- Country wise Analysis

```
top_countries = df_country.sort_values('Confirmed',ascending=False).head(10)
top_countries
```

	Country	Confirmed	Recovered	Deaths
173	US	224345948	56353416	11011411
23	Brazil	89524967	54492873	3938034
138	Russia	45408411	25120448	619385
79	India	40883464	23783720	1111831
157	Spain	27404045	15093583	3033030
177	United Kingdom	26748587	126217	3997775
85	Italy	26745145	15673910	3707717
61	France	21210926	7182115	3048524
65	Germany	21059152	17107839	871322
81	Iran	19339267	15200895	1024136

Next steps: Generate code with `top_countries` New interactive sheet

```
fig = px.bar(top_countries,x='Country',y='Confirmed',color='Country',template='plotly_dark',title='Top 10 Countries with Hi
fig.show()
```



Step 6:- Feature Engineering

```
# Daily New Cases
df_global['New_Cases'] = df_global['Confirmed'].diff()
df_global['New_Cases']
```

	New_Cases
0	NaN
1	99.0
2	287.0
3	493.0
4	684.0
...	...
183	282756.0
184	281164.0
185	255545.0
186	204606.0
187	228689.0

188 rows × 1 columns

dtype: float64

```
# Recovery Rate
df_global['Recovery_Rate'] = df_global['Recovered'] / df_global['Confirmed']
df_global['Recovery_Rate']
```

	Recovery_Rate
0	0.050450
1	0.045872
2	0.038257
3	0.027197
4	0.024551
...	...
183	0.561618
184	0.566103
185	0.570738
186	0.571842
187	0.574503

188 rows × 1 columns

dtype: float64

```
# Death Rate
df_global['Death_Rate'] = df_global['Deaths'] / df_global['Confirmed']
df_global['Death_Rate']
```

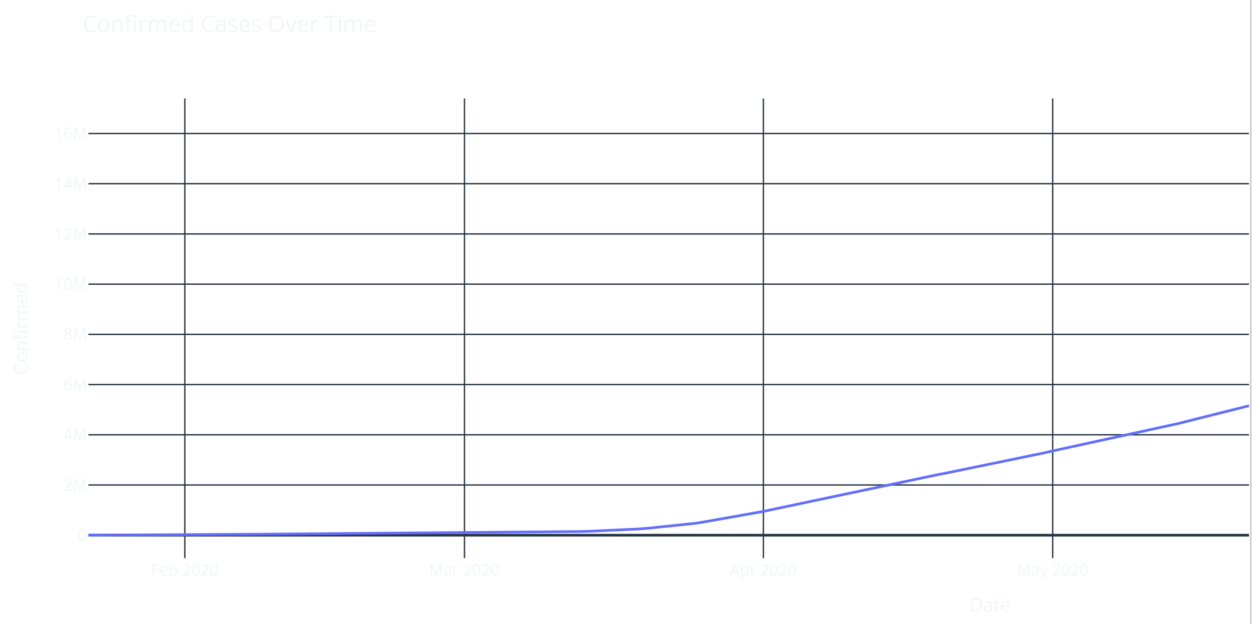
	Death_Rate
0	0.030631
1	0.027523
2	0.027630
3	0.029289
4	0.026440
...	...
183	0.040844
184	0.040506
185	0.040164
186	0.039911
187	0.039685

188 rows × 1 columns

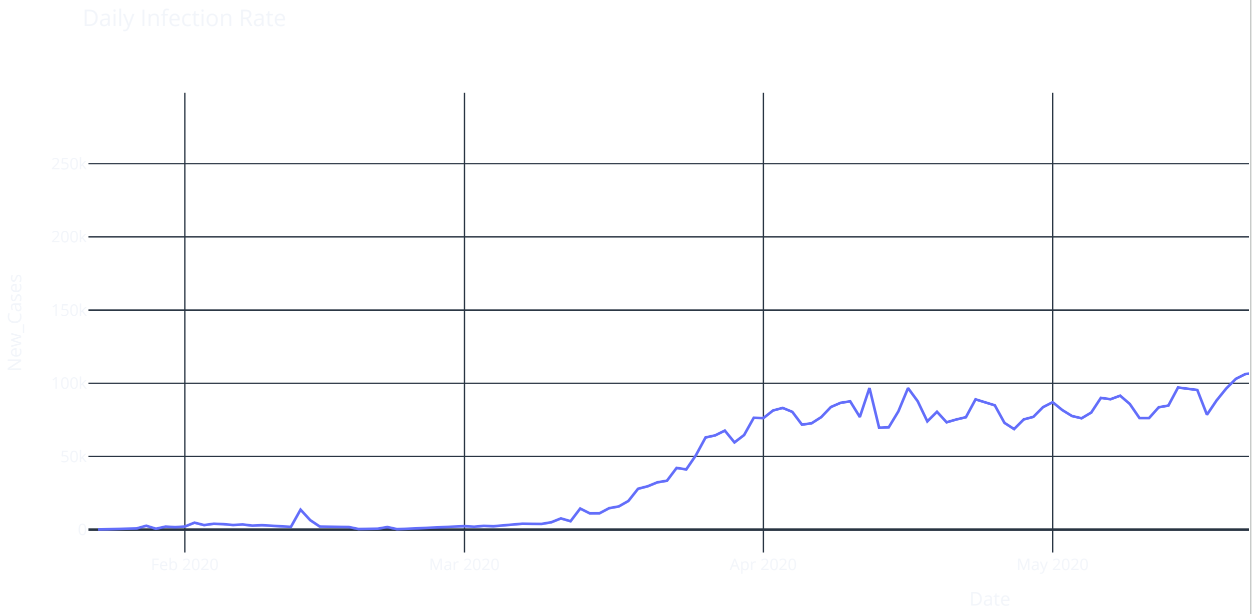
dtype: float64

Step 7:- Time Trend Visualization

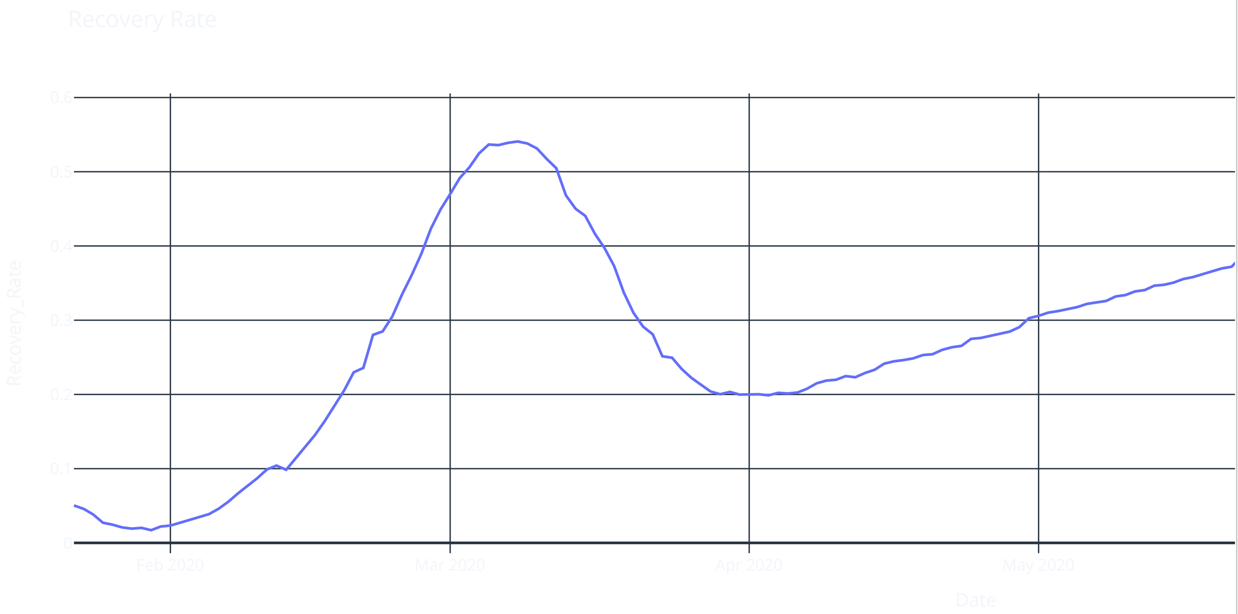
```
# Confirmed Cases
fig = px.line(df_global,x='Date',y='Confirmed',template='plotly_dark',title='Confirmed Cases Over Time')
fig.show()
```



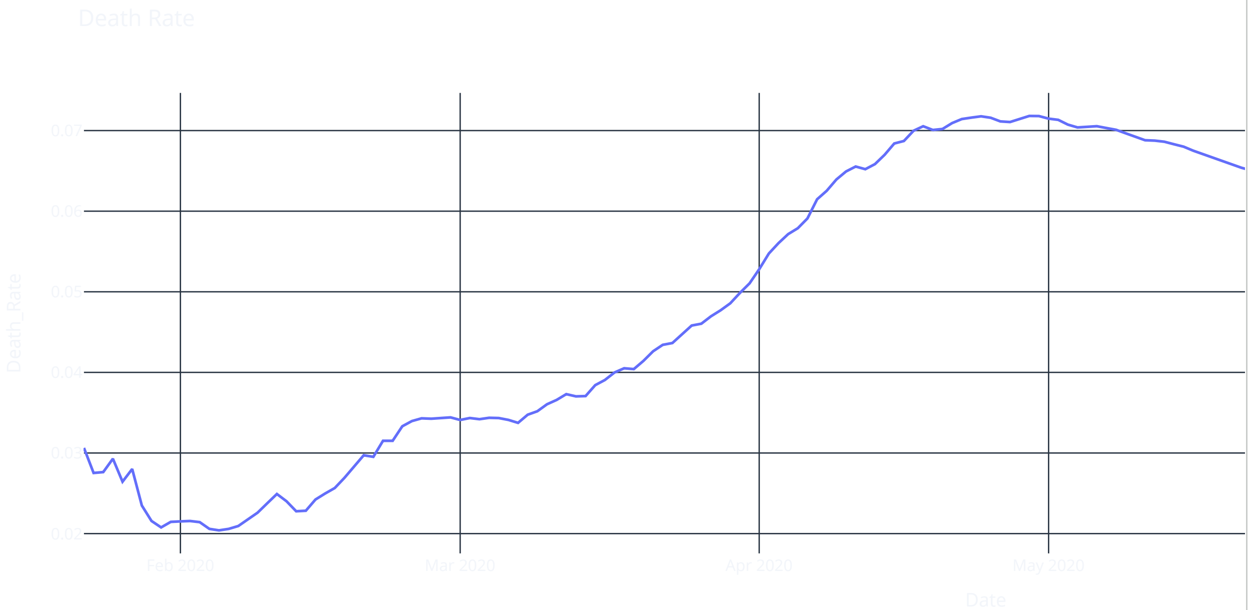

```
# Infection Rate
fig = px.line(df_global,x='Date',y='New_Cases',template='plotly_dark',title='Daily Infection Rate')
fig.show()
```




```
# Recovery Rate
fig = px.line(df_global,x='Date',y='Recovery_Rate',template='plotly_dark',title='Recovery Rate')
fig.show()
```




```
# Death Rate
fig = px.line(df_global,x='Date',y='Death_Rate',template='plotly_dark',title='Death Rate')
fig.show()
```



Step 8:- Moving Average

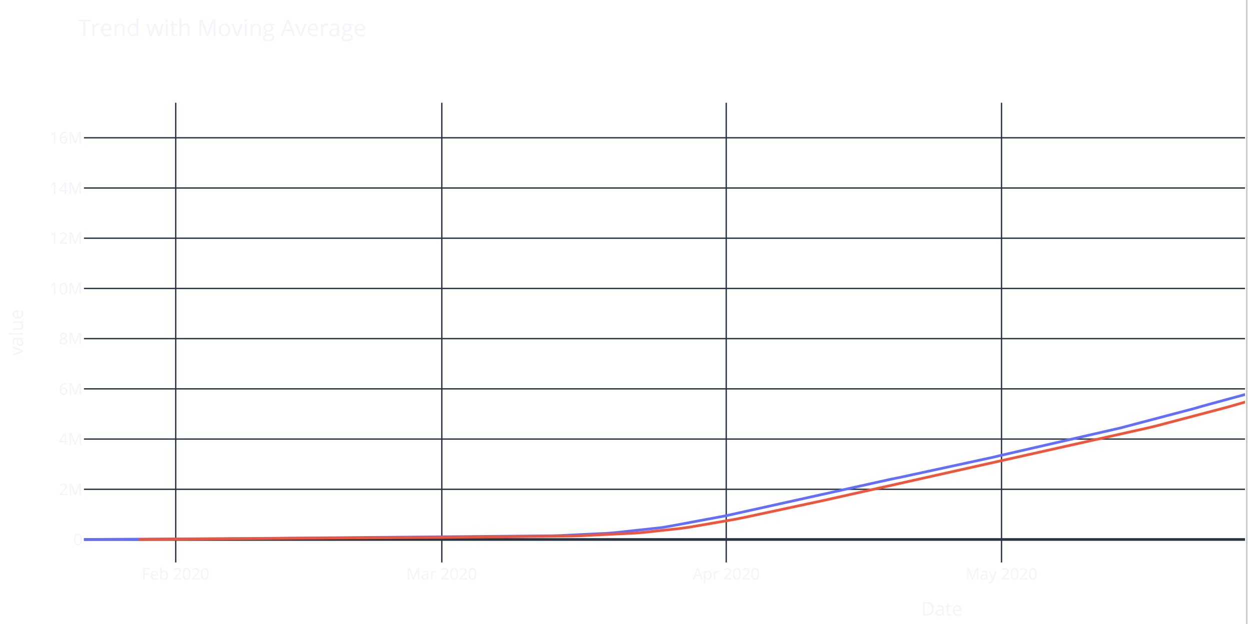
```
df_global['MA_7'] = df_global['Confirmed'].rolling(7).mean()
df_global['MA_7']
```

	MA_7
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN
...	...
183	1.475036e+07
184	1.499851e+07
185	1.524923e+07
186	1.549851e+07
187	1.575091e+07

188 rows × 1 columns

dtype: float64

```
# Plot
fig = px.line(df_global,x='Date',y=['Confirmed','MA_7'],template='plotly_dark',title='Trend with Moving Average')
fig.show()
```



Step 9:- Prepare Data for Prophet

```
prophet_df = df_global[['Date','Confirmed']]
prophet_df.columns = ['ds','y']
prophet_df
```

	ds	y
0	2020-01-22	555
1	2020-01-23	654
2	2020-01-24	941
3	2020-01-25	1434
4	2020-01-26	2118
...	...	...
183	2020-07-23	15510481
184	2020-07-24	15791645
185	2020-07-25	16047190
186	2020-07-26	16251796
187	2020-07-27	16480485

188 rows × 2 columns

Next steps: Generate code with prophet_df New interactive sheet

Step 10:- Train Model

```
model = Prophet()
model.fit(prophet_df)
```

```
INFO:prophet:Disabling yearly seasonality. Run prophet with yearly_seasonality=True to override this.
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
<prophet.forecaster.Prophet at 0x789f53055ee0>
```

Step 11:- Predict Future (7 Days)

```
future = model.make_future_dataframe(periods=7)
forecast = model.predict(future)
forecast
```

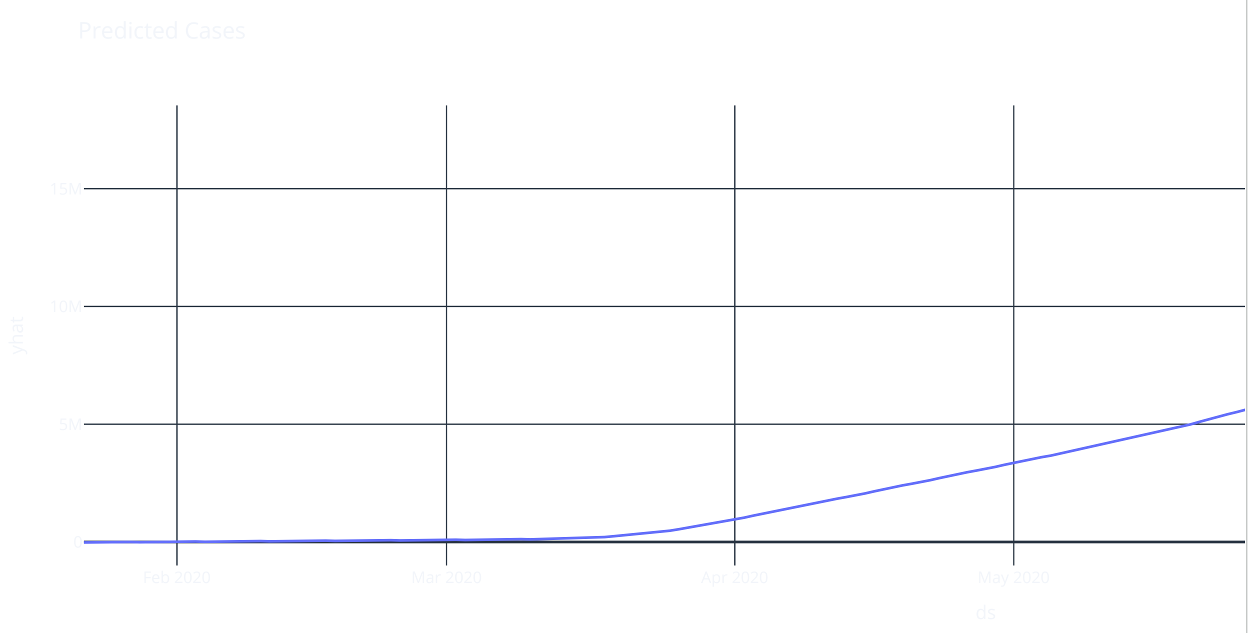
	ds	trend	yhat_lower	yhat_upper	trend_lower	trend_upper	additive_terms	additive_terms_lower	addi
0	2020-01-22	-9.613288e+03	-1.292410e+05	8.589034e+04	-9.613288e+03	-9.613288e+03	-11063.558307	-11063.558307	
1	2020-01-23	-6.933409e+03	-1.095000e+05	9.572055e+04	-6.933409e+03	-6.933409e+03	-1117.543863	-1117.543863	
2	2020-01-24	-4.253530e+03	-9.408092e+04	1.141530e+05	-4.253530e+03	-4.253530e+03	10080.978737	10080.978737	
3	2020-01-25	-1.573651e+03	-9.221896e+04	1.206262e+05	-1.573651e+03	-1.573651e+03	13750.326871	13750.326871	
4	2020-01-26	1.106228e+03	-9.318889e+04	1.108914e+05	1.106228e+03	1.106228e+03	7298.791978	7298.791978	
...	...	...	...	...	...	...	...	...	
190	2020-07-30	1.674503e+07	1.663193e+07	1.684878e+07	1.673843e+07	1.675066e+07	-1117.543863	-1117.543863	
191	2020-07-31	1.694902e+07	1.685528e+07	1.706586e+07	1.693561e+07	1.696256e+07	10080.978737	10080.978737	
192	2020-08-01	1.715301e+07	1.705768e+07	1.727059e+07	1.713185e+07	1.717377e+07	13750.326871	13750.326871	
193	2020-08-02	1.735701e+07	1.725081e+07	1.747781e+07	1.732709e+07	1.738657e+07	7298.791978	7298.791978	
194	2020-08-03	1.756100e+07	1.743453e+07	1.767565e+07	1.751650e+07	1.760184e+07	-2102.755455	-2102.755455	

195 rows × 16 columns

Next steps: Generate code with forecast New interactive sheet

Step 12:- Visualize Prediction

```
fig = px.line(forecast,x='ds',y='yhat',template='plotly_dark',title='Predicted Cases')
fig.show()
```

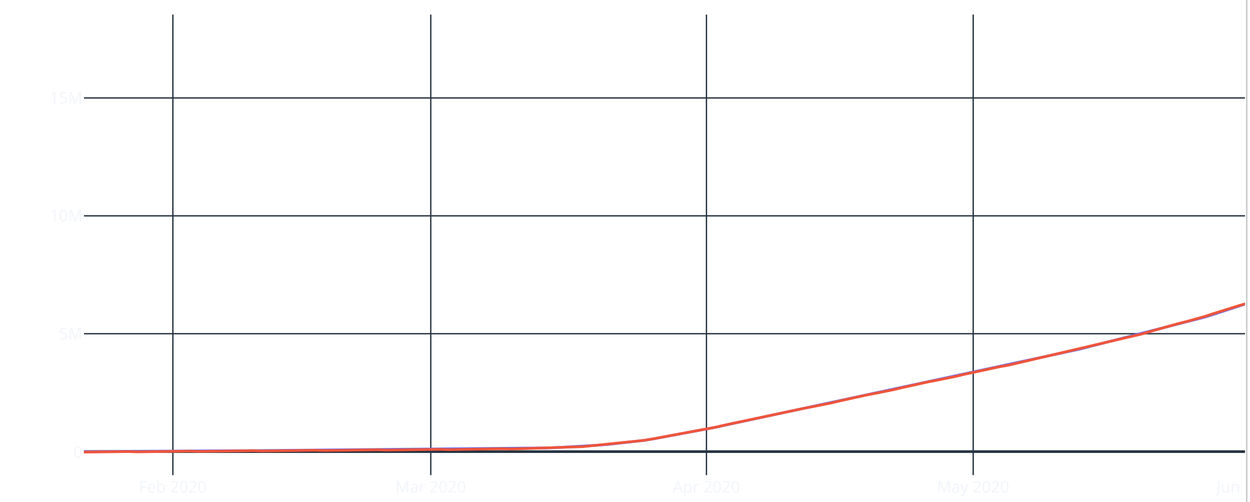


Step 13:- Actual vs Predicted

```
fig = go.Figure()

fig.add_trace(go.Scatter(x=df_global['Date'], y=df_global['Confirmed'], name='Actual'))
fig.add_trace(go.Scatter(x=forecast['ds'], y=forecast['yhat'], name='Predicted'))
fig.update_layout(template='plotly_dark')

fig.show()
```



Step 14 :- Seasonality Visualization

```
model.plot_components(forecast)
```